

- 1. main.cpp: 程序总流程
- 2. settings.cpp: 菜单设置转成游戏参数
- 3. snake_logic.cpp: A 负责的前半基础部分
- 4. crazy_mode.cpp: A 负责的前半部分
- 5. loading.cpp: Loading 过渡
- 6. score_file.cpp: 最高分文件读写
- 7. draw_ui.cpp: 菜单和游戏 UI

A 同学代码阅读顺序（含代码）

说明：本文档把 A 同学负责阅读的代码按推荐顺序整理到一起。snake_logic.cpp 和 crazy_mode.cpp 只放 A 负责的部分，其它 A 负责文件放完整代码。

1. main.cpp: 程序总流程

来源文件: main.cpp

```
#include "assets.h"
#include "constants.h"
#include "crazy_mode.h"
#include "draw_scene.h"
#include "draw_ui.h"
#include "input.h"
#include "loading.h"
#include "score_file.h"
#include "settings.h"
#include "snake_logic.h"

#include <cstdlib>
#include <ctime>

// “启动、更新、绘制、关闭”四个主流程。

// 函数作用：初始化窗口、资源、默认设置和游戏初始状态。
void InitProgram(GameContext &context)
{
    srand((unsigned int)time(nullptr)); // 用当前时间做随机种子。

    // raylib 的窗口初始化。
    SetConfigFlags(FLAG_MSAA_4X_HINT); // 开启抗锯齿。
    InitWindow(SCREEN_WIDTH, SCREEN_HEIGHT, "3D Snake Game");
    SetTargetFPS(60);

    UpdateCameraPosition(context);
    LoadAssets(context); // 背景图片加载。
```

```

    context.highScore = LoadHighScore();
    context.loading.timer = 0.0f; // Loading 计时器从 0 开始，表示启动时不显示
Loading。

    // 设置默认关卡、速度、难度和模式。
    SetDefaultSettings(context);

    context.game.status = LevelSelect; // 显示菜单。
}

// 函数作用：每一帧更新输入、Loading、游戏规则和疯狂模式。
void UpdateProgram(GameContext &context)
{
    HandleInput(context); // 先读输入。
    UpdateLoading(context);
    UpdateGame(context);
    UpdateEffects(context); // 视觉效果。
    UpdateCrazyMode(context);
}

// 函数作用：每一帧绘制背景、3D 场景、UI 和 Loading 覆盖层。
void DrawProgram(GameContext &context)
{
    BeginDrawing(); // raylib 要求所有绘制都写在
BeginDrawing 和 EndDrawing 之间。
    ClearBackground((Color){20, 25, 40, 255}); // 默认底色。

    if (context.game.status != LevelSelect)
    {
        // 游戏运行界面先画背景图，再进入 3D 模式画地图和蛇。
        DrawBackgroundImage(context);

        BeginMode3D(context.camera);
        DrawGame3D(context);
        EndMode3D();
    }

    // UI 永远最后画，这样文字和菜单会盖在 3D 场景上面。
    DrawUI(context);
    DrawLoadingScreen(context);

    EndDrawing();
}

// 函数作用：程序退出前释放资源并关闭窗口。
void CloseProgram(GameContext &context)
{
    UnloadAssets(context); // 释放手动加载的图片资源。
    CloseWindow();
}

int main()
{
    // GameContext 是“游戏总数据包”。
    GameContext context = {};

    InitProgram(context);

```

```
// raylib 的主循环。  
// 只要窗口没有被关闭，就一直更新并绘制。  
while (!WindowShouldClose())  
{  
    UpdateProgram(context);  
    DrawProgram(context);  
}  
  
CloseProgram(context);  
  
return 0;  
}
```

2. settings.cpp: 菜单设置转成游戏参数

来源文件: `settings.cpp`

```
#include "settings.h"  
#include "constants.h"  
  
// settings.cpp 负责“菜单选项 -> 实际游戏参数”。  
// 例如玩家选 Level 3，这里会把地图大小设置为 20 x 20。  
  
// 函数作用：计算主菜单左上角的 x 坐标，让菜单水平居中。  
int GetMenuX()  
{  
    // 菜单水平居中。  
    return (SCREEN_WIDTH - MENU_WIDTH) / 2; // 屏幕宽度减菜单宽度，再除以 2 就是居中  
    x。  
}  
  
// 函数作用：根据关卡编号返回地图边长。  
int GetMapSizeByLevel(int level)  
{  
    // 关卡只决定地图大小，不决定速度和目标分数。  
    if (level == 1)  
    {  
        return 10; // Level 1: 10 x 10, 小地图。  
    }  
  
    if (level == 2)  
    {  
        return 15; // Level 2: 15 x 15。  
    }  
  
    if (level == 3)  
    {  
        return 20; // Level 3: 20 x 20。  
    }  
}
```

```

    if (level == 4)
    {
        return 25; // Level 4: 25 x 25, 大地图。
    }

    return DEFAULT_GRID_SIZE; // 非法关卡使用默认地图大小兜底。
}

// 函数作用：返回当前关卡的显示文字。
std::string GetLevelName(const GameContext &context)
{
    return "Level " + std::to_string(context.settings.currentLevel); // 把数字关卡拼成显示文本。
}

// 函数作用：设置当前关卡，并同步地图大小和目标分。
void SetLevel(GameContext &context, int level)
{
    // 如果传入错误关卡，就回到 Level 1。
    if (level < 1)
    {
        level = 1; // 小于 1 的非法值改回 Level 1。
    }

    if (level > LEVEL_COUNT)
    {
        level = 1; // 超过关卡数量的非法值也改回 Level 1。
    }

    context.settings.currentLevel = level; // 保存当前关卡编号。
    context.settings.gridSize = GetMapSizeByLevel(level); // 根据关卡编号更新地图大小。

    UpdateTargetScore(context); // 地图大小变了，目标分也要跟着变。
}

// 函数作用：根据速度等级设置蛇的移动速度参数。
void SetSpeedByLevel(GameContext &context, int speedLevel)
{
    // 先把速度等级限制在合法范围内。
    if (speedLevel < MIN_SPEED_LEVEL)
    {
        speedLevel = MIN_SPEED_LEVEL; // 速度不能低于最小档。
    }

    if (speedLevel > MAX_SPEED_LEVEL)
    {
        speedLevel = MAX_SPEED_LEVEL; // 速度不能高于最大档。
    }

    context.settings.speedLevel = speedLevel; // 保存菜单上显示的速度档位。

    // 不同速度等级对应不同移动间隔。
    // 间隔越小，蛇移动越快。
    if (speedLevel == 1)
    {
        context.settings.speed = {0.28f, 0.14f, 0.006f}; // 慢速：起步慢，最低速度也
    }
}

```

较慢。

```
}  
else if (speedLevel == 2)  
{  
    context.settings.speed = {0.22f, 0.11f, 0.008f}; // 较慢。  
}  
else if (speedLevel == 3)  
{  
    context.settings.speed = {0.18f, 0.08f, 0.010f}; // 默认速度。  
}  
else if (speedLevel == 4)  
{  
    context.settings.speed = {0.14f, 0.07f, 0.012f}; // 较快。  
}  
else  
{  
    context.settings.speed = {0.10f, 0.05f, 0.014f}; // 快速：移动间隔最短。  
}  
}
```

// 函数作用：根据地图大小和难度等级计算目标分数。

```
void UpdateTargetScore(GameContext &context)  
{  
    // 目标分数和地图大小、难度等级有关。  
    context.settings.targetScore =  
        context.settings.gridSize * // 地图越大，目标分越高。  
        context.settings.difficultyLevel * // 难度越高，目标分越高。  
        2; // 简单倍率，避免目标分计算太复杂。  
}
```

// 函数作用：设置游戏启动时的默认菜单选项。

```
void SetDefaultSettings(GameContext &context)  
{  
    // 游戏启动时的默认菜单设置。  
    context.settings.currentLevel = 1; // 默认选 Level 1。  
    context.settings.difficultyLevel = 3; // 默认中等难度。  
    context.settings.crazyMode = false; // 默认普通模式。  
  
    SetLevel(context, 1); // 同步地图大小和目标分。  
    SetSpeedByLevel(context, 3); // 默认速度等级 3。  
}
```

3. snake_logic.cpp：A 负责的前半基础部分

来源文件：snake_logic.cpp

```
#include "snake_logic.h"  
#include "constants.h"  
#include "score_file.h"
```

```

#include "settings.h"
#include <cstdlib>

// snake_logic.cpp 是贪吃蛇“规则”的核心文件。
// 这里负责移动、吃食物、增长、碰撞、生成食物。

// 函数作用：初始化或重新开始一局游戏。
void InitGame(GameContext &context)
{
    // 重新开始一局时，先清空旧蛇身。
    context.game.snake.clear(); // 清掉上一局遗留的身体格子。

    // 蛇出生。
    Cell start;
    start.x = context.settings.gridSize / 2; // x 放在地图中间。
    start.z = context.settings.gridSize / 2;

    context.game.snake.push_back(start); // 第 0 节永远是蛇头。。
    context.game.snake.push_back({start.x - 1, start.z});
    context.game.snake.push_back({start.x - 2, start.z});

    // 初始方向向右。
    context.game.direction = {1, 0}; // {1, 0} 表示 x 增加，也就是向右。
    context.game.nextDirection = context.game.direction; // 还没输入时，下一方向等于当前方向。

    // 重置一局游戏的基本状态。
    context.game.food = {-1, -1}; // -1 表示暂时没有有效食物。
    context.game.status = Playing;
    context.game.score = 0;
    context.game.moveTimer = 0.0f;
    context.game.started = false;
    context.game.isSuperFood = false;
    context.game.growLeft = 0; // 一开始没有待增长次数。

    // 重置临时视觉效果。
    context.effects.eatTimer = 0.0f; // 吃食物爆炸效果关闭。
    context.effects.scoreFlashTimer = 0.0f; // 分数闪烁效果关闭。
    context.effects.lastFoodCell = {-1, -1}; // 没有上一次食物位置。

    // 重置疯狂模式数据。
    // 即使普通模式也重置，保证每局开始状态干净。
    context.crazy.blocks.clear(); // 删除上一局留下的蓝色方块。
    context.crazy.eventTimer = 3.0f; // 开局约 3 秒后才触发第一次疯狂事件。
    context.crazy.boostTimer = 0.0f; // 当前不处于加速中。
    context.crazy.redFlashTimer = 0.0f; // 当前没有红光预警。
    context.crazy.greenFlashTimer = 0.0f; // 当前没有绿光恢复提示。
    context.crazy.boostWaiting = false; // 当前没有“红光结束后加速”的等待状态。

    // 第一颗食物在所有数据重置后生成。
    SpawnFood(context); // 初始化完蛇身后再生成食物，避免食物刷在蛇身上。
}

// 函数作用：使用菜单里的当前设置开始游戏。
void StartSelectedGame(GameContext &context)
{

```

```

// 从菜单进入游戏时，先应用菜单设置，再初始化一局游戏。
SetLevel(context, context.settings.currentLevel);
SetSpeedByLevel(context, context.settings.speedLevel);
UpdateTargetScore(context);
InitGame(context);
}

// 函数作用：每帧更新普通贪吃蛇移动逻辑。
void UpdateGame(GameContext &context)
{
    // 菜单、暂停、失败、胜利时不移动蛇。
    bool gameIsPlaying = context.game.status == Playing; // 只有 Playing 才允许移动。

    // 玩家还没按方向键时，蛇先停在原地。
    bool snakeHasStarted = context.game.started; // false 表示玩家还没按过方向键。

    if (!gameIsPlaying || !snakeHasStarted)
    {
        return;
    }

    context.game.moveTimer += GetFrameTime(); // 累加本帧经过的秒数。

    // moveTimer 累计到移动间隔后，蛇才移动一格。
    // 这样可以使游戏保持“按格子移动”，更容易理解。
    if (context.game.moveTimer >= GetCurrentMoveInterval(context))
    {
        context.game.moveTimer = 0.0f; // 走完一格后重新计时。
        MoveSnake(context);           // 真正移动蛇一格。
    }
}

// 函数作用：更新吃食物爆炸和分数闪烁的倒计时。
void UpdateEffects(GameContext &context)
{
    // 这里统一更新临时视觉效果倒计时。
    // 倒计时减到 0 后，对应效果就不再绘制。
    float deltaTime = GetFrameTime(); // 本帧耗时，用它减少倒计时。

    if (context.effects.eatTimer > 0.0f)
    {
        context.effects.eatTimer -= deltaTime; // 倒计时越减越小，效果自然结束。

        if (context.effects.eatTimer < 0.0f)
        {
            context.effects.eatTimer = 0.0f; // 防止倒计时出现负数。
        }
    }

    if (context.effects.scoreFlashTimer > 0.0f)
    {
        context.effects.scoreFlashTimer -= deltaTime; // 分数闪烁也用同样的倒计时写法。

        if (context.effects.scoreFlashTimer < 0.0f)
        {

```

```

        context.effects.scoreFlashTimer = 0.0f; // 小于 0 后统一压回 0。
    }
}

// 函数作用：判断两个格子坐标是否相同。
bool IsSameCell(Cell a, Cell b)
{
    // 两个格子的 x 和 z 都相同，才算同一个格子。
    return a.x == b.x && a.z == b.z; // 两个方向都相等，才是同一个格子。
}

// 函数作用：判断指定格子是否在蛇身上。
bool IsCellOnSnake(const GameContext &context, Cell cell)
{
    for (int i = 0; i < (int)context.game.snake.size(); i++)
    {
        if (IsSameCell(context.game.snake[i], cell))
        {
            return true;
        }
    }

    return false;
}

// 函数作用：判断指定格子是否有蓝色方块。
bool IsCellOnCrazyBlock(const GameContext &context, Cell cell)
{
    // 生成食物时，只要这个格子有蓝方块，就不能放食物。
    for (int i = 0; i < (int)context.crazy.blocks.size(); i++)
    {
        if (IsSameCell(context.crazy.blocks[i].cell, cell))
        {
            return true;
        }
    }

    return false;
}

// 函数作用：判断指定格子是否有实体蓝色障碍。
bool IsCellOnSolidCrazyBlock(const GameContext &context, Cell cell)
{
    // 只有 solid 为 true 的蓝方块才会撞死蛇。
    for (int i = 0; i < (int)context.crazy.blocks.size(); i++)
    {
        bool blockIsSolid = context.crazy.blocks[i].solid; // true 表示已经变成会撞死
的实体。
        bool sameCell = IsSameCell(context.crazy.blocks[i].cell, cell);

        if (blockIsSolid && sameCell)
        {
            return true;
        }
    }
}

```



```

    return false;
}

// 函数作用：判断蛇头是否撞到地图边界。
bool CheckWallCollision(const GameContext &context, Cell head)
{
    // 坐标小于 0 或大于等于地图大小，就说明蛇头出了地图。

```

4. crazy_mode.cpp: A 负责的前半部分

来源文件：crazy_mode.cpp

```

#include "crazy_mode.h"
#include "constants.h"
#include "snake_logic.h"
#include <cstdlib>

// crazy_mode.cpp 只负责疯狂模式的额外规则。
// 普通贪吃蛇规则仍然在 snake_logic.cpp 中。
// 这里主要是几个计时器：红光预警、加速、绿光恢复、蓝色方块。

// 函数作用：疯狂模式总入口，每帧更新所有疯狂模式事件。
void UpdateCrazyMode(GameContext &context)
{
    // 没开疯狂模式，或者游戏不在进行中，就不更新疯狂模式。
    bool crazyModeIsOff = !context.settings.crazyMode; // 菜单没有开启疯狂模式。
    bool gameIsNotPlaying = context.game.status != Playing; // 暂停、失败、胜利、菜单
    都不更新事件。

    if (crazyModeIsOff || gameIsNotPlaying)
    {
        return;
    }

    float deltaTime = GetFrameTime(); // 本帧经过的秒数，所有倒计时都减它。

    // 一个总入口拆成三个小步骤，比写在一个大函数里更清楚。
    UpdateBoostEvent(context, deltaTime); // 更新红光、加速、绿光。
    UpdateBlueBlocks(context, deltaTime); // 更新蓝方块预警是否结束。
    TryStartCrazyEvent(context); // 看看是否该触发下一个随机事件。
}

// 函数作用：更新红光预警、加速和绿光恢复三个计时器。
void UpdateBoostEvent(GameContext &context, float deltaTime)
{
    // boostTimer > 0 表示正在加速。
    // 倒计时结束后，进入绿色恢复提示。
    if (context.crazy.boostTimer > 0.0f)
    {
        context.crazy.boostTimer -= deltaTime; // 加速剩余时间逐帧减少。
    }
}

```

```

    if (context.crazy.boostTimer <= 0.0f)
    {
        context.crazy.boostTimer = 0.0f; // 防止加速时间变成负数。
        context.crazy.greenFlashTimer = GREEN_FLASH_TIME; // 加速结束后显示绿色恢
复提示。
    }
}

// redFlashTimer 是加速前的红色预警。
// 红光结束后，才真正开始加速。
if (context.crazy.redFlashTimer > 0.0f)
{
    context.crazy.redFlashTimer -= deltaTime; // 红光预警剩余时间逐帧减少。

    if (context.crazy.redFlashTimer <= 0.0f)
    {
        context.crazy.redFlashTimer = 0.0f; // 红光结束。

        if (context.crazy.boostWaiting)
        {
            context.crazy.boostTimer = BOOST_TIME; // 红光结束后正式进入加速。
            context.crazy.boostWaiting = false; // 等待状态用完后关闭。
        }
    }
}

// greenFlashTimer 是加速结束后的恢复提示。
if (context.crazy.greenFlashTimer > 0.0f)
{
    context.crazy.greenFlashTimer -= deltaTime; // 绿色恢复提示逐帧减少。

    if (context.crazy.greenFlashTimer < 0.0f)
    {
        context.crazy.greenFlashTimer = 0.0f; // 防止出现负数。
    }
}
}

// 函数作用：更新蓝色方块从闪烁预警到实体障碍的过程。
void UpdateBlueBlocks(GameContext &context, float deltaTime)
{
    // 蓝色方块刚出现时先闪烁，不会立刻变成实体。
    // flashTimer 结束后，solid 变成 true，蛇撞到才会失败。
    for (int i = 0; i < (int)context.crazy.blocks.size(); i++)
    {
        if (!context.crazy.blocks[i].solid) // 只有预警方块需要倒计时。
        {
            context.crazy.blocks[i].flashTimer -= deltaTime; // 蓝方块闪烁预警剩余时
间。

            if (context.crazy.blocks[i].flashTimer <= 0.0f)
            {
                context.crazy.blocks[i].flashTimer = 0.0f; // 预警时间结束。
                context.crazy.blocks[i].solid = true; // 变成实体障碍，之后撞到会失
败。
            }
        }
    }
}

```

```
}  
}
```

5. loading.cpp: Loading 过渡

来源文件: `loading.cpp`

```
#include "loading.h"  
#include "constants.h"  
  
// Loading 不是一个新的 GameStatus。  
// 它只是一个覆盖在画面最上层的倒计时黑屏。  
  
// 函数作用: 启动一次 Loading 黑屏过渡。  
void StartLoading(GameContext &context)  
{  
    // 重新把倒计时设为完整 Loading 时长。  
    context.loading.timer = LOADING_TIME; // 总时长 = 黑屏保持时间 + 渐隐时间。  
}  
  
// 函数作用: 每帧减少 Loading 倒计时。  
void UpdateLoading(GameContext &context)  
{  
    // 每帧减少一点时间, 减到 0 就结束 Loading。  
    if (context.loading.timer > 0.0f)  
    {  
        context.loading.timer -= GetFrameTime(); // 每帧扣掉经过的秒数。  
  
        if (context.loading.timer < 0.0f)  
        {  
            context.loading.timer = 0.0f; // 小于 0 时统一归零, 方便 IsLoading 判断。  
        }  
    }  
}  
  
// 函数作用: 判断当前是否还处于 Loading 过渡中。  
bool IsLoading(const GameContext &context)  
{  
    // 只要 timer 大于 0, 就说明 Loading 还在显示。  
    return context.loading.timer > 0.0f; // 只要倒计时没结束, 就还在 Loading。  
}  
  
// 函数作用: 绘制 Loading 黑屏和文字, 并实现渐隐。  
void DrawLoadingScreen(const GameContext &context)  
{  
    if (!IsLoading(context))  
    {  
        return;  
    }  
  
    // Loading 分两段:
```

```

// 先保持纯黑，再逐渐透明。
float timePassed = LOADING_TIME - context.loading.timer; // 已经过去的 Loading
时间。
float alphaRate;

if (timePassed < LOADING_HOLD_TIME)
{
    alphaRate = 1.0f; // 前半段保持完全黑屏。
}
else
{
    alphaRate = context.loading.timer / LOADING_FADE_OUT_TIME; // 后半段从 1 慢
慢降到 0。
}

// alpha 控制黑屏透明度。
// 255 是完全不透明，0 是完全透明。
unsigned char alpha = (unsigned char)(255 * alphaRate); // 把 0-1 的比例转成 0-
255 的透明度。

DrawRectangle(
    0,
    0,
    SCREEN_WIDTH,
    SCREEN_HEIGHT,
    (Color){0, 0, 0, alpha});

const char *text = "LOADING..."; // Loading 显示的文字。
int fontSize = 46; // 文字大小。
int textWidth = MeasureText(text, fontSize); // 用于水平居中。

DrawText(
    text,
    (SCREEN_WIDTH - textWidth) / 2,
    SCREEN_HEIGHT / 2 - fontSize / 2,
    fontSize,
    (Color){255, 255, 255, alpha});
}

```

6. score_file.cpp：最高分文件读写

来源文件：score_file.cpp

```

#include "score_file.h"
#include "constants.h"
#include <fstream>

// score_file.cpp 只负责最高分文件。
// 文件读写和游戏规则分开，代码更容易找。

// 函数作用：从文件读取历史最高分。

```

```

int LoadHighScore()
{
    // 文件不存在时，默认最高分就是 0。
    int savedScore = 0; // 兜底值：没有文件或读取失败时用 0。

    std::ifstream inputFile(HIGH_SCORE_FILE); // 打开最高分文件用于读取。

    if (inputFile.is_open())
    {
        inputFile >> savedScore; // 文件里只保存一个整数。
        inputFile.close(); // 读完手动关闭文件。
    }

    return savedScore; // 返回读取到的最高分。
}

// 函数作用：把当前最高分保存到文件。
void SaveHighScore(const GameContext &context)
{
    // 每次保存都会覆盖原文件，只保留一个最高分数字。
    std::ofstream outputFile(HIGH_SCORE_FILE); // 打开文件用于写入，会覆盖旧内容。

    if (outputFile.is_open())
    {
        outputFile << context.highScore; // 只写入最高分数字。
        outputFile.close(); // 写完关闭文件。
    }
}

// 函数作用：当前分数超过最高分时更新并保存。
void UpdateHighScore(GameContext &context)
{
    // 只有当前分数超过最高分，才写文件。
    if (context.game.score > context.highScore)
    {
        context.highScore = context.game.score; // 内存里的最高分先更新。
        SaveHighScore(context); // 再保存到文件。
    }
}

```

7. draw_ui.cpp：菜单和游戏 UI

来源文件：draw_ui.cpp

```

#include "draw_ui.h"
#include "assets.h"
#include "constants.h"
#include "settings.h"

```

// 函数作用：计算某个关卡按钮在屏幕上的矩形区域。

```

Rectangle GetLevelButtonRect(int index)
{
    // index 从 0 开始, 所以 index 0 对应 Level 1。
    int buttonWidth = 155; // 单个关卡按钮宽度。
    int buttonHeight = 65; // 单个关卡按钮高度。
    int buttonGap = 25; // 关卡按钮之间的间距。

    int totalWidth =
        buttonWidth * LEVEL_COUNT + buttonGap * (LEVEL_COUNT - 1);

    int startX = GetMenuX() + (MENU_WIDTH - totalWidth) / 2; // 让这一排按钮在菜单里
    居中。
    int y = MENU_Y + 190; // 关卡按钮所在的纵向位置。

    return {
        (float)(startX + index * (buttonWidth + buttonGap)), // 第 index 个按钮的
    x。
        (float)y, // 按钮 y。
        (float)buttonWidth, // 按钮宽度。
        (float)buttonHeight
    };
}

// 函数作用: 返回速度减号按钮的矩形区域。
Rectangle GetSpeedMinusButtonRect()
{
    return {
        (float)(GetMenuX() + 110),
        (float)(MENU_Y + 400),
        65.0f,
        50.0f};
}

// 函数作用: 返回速度加号按钮的矩形区域。
Rectangle GetSpeedPlusButtonRect()
{
    return {
        (float)(GetMenuX() + MENU_WIDTH - 175),
        (float)(MENU_Y + 400),
        65.0f,
        50.0f};
}

// 函数作用: 返回难度减号按钮的矩形区域。
Rectangle GetDifficultyMinusButtonRect()
{
    return {
        (float)(GetMenuX() + 110),
        (float)(MENU_Y + 550),
        65.0f,
        50.0f};
}

// 函数作用: 返回难度加号按钮的矩形区域。
Rectangle GetDifficultyPlusButtonRect()
{
    return {

```

```

        (float)(GetMenuX() + MENU_WIDTH - 175),
        (float)(MENU_Y + 550),
        65.0f,
        50.0f});
}

```

// 函数作用：返回疯狂模式开关按钮的矩形区域。

```

Rectangle GetCrazyModeButtonRect()
{
    int buttonWidth = 260; // Crazy Mode 按钮宽度。
    int buttonHeight = 55; // Crazy Mode 按钮高度。

    return {
        (float)(GetMenuX() + (MENU_WIDTH - buttonWidth) / 2),
        (float)(MENU_Y + 615),
        (float)buttonWidth,
        (float)buttonHeight};
}

```

// 函数作用：返回 START 按钮的矩形区域。

```

Rectangle GetStartButtonRect()
{
    int buttonWidth = 240; // START 按钮宽度。
    int buttonHeight = 70; // START 按钮高度。

    return {
        (float)(GetMenuX() + (MENU_WIDTH - buttonWidth) / 2),
        (float)(MENU_Y + 680),
        (float)buttonWidth,
        (float)buttonHeight};
}

```

// 函数作用：判断鼠标是否点击了指定按钮区域。

```

bool IsButtonClicked(Rectangle button)
{
    // 鼠标在按钮矩形内，并且按下左键，才算点击按钮。
    bool mouseOnButton = CheckCollisionPointRec(GetMousePosition(), button); // 鼠标坐标是否在按钮矩形内。
    bool leftMousePressed = IsMouseButtonPressed(MOUSE_BUTTON_LEFT); // 左键是否刚按下。

    return mouseOnButton && leftMousePressed;
}

```

// 函数作用：绘制一个通用菜单按钮。

```

void DrawButton(Rectangle button, const char *text, bool selected)
{
    // 一个通用按钮函数。
    // selected 用来表示当前选中的关卡或模式。
    bool mouseOver = CheckCollisionPointRec(GetMousePosition(), button); // 用于悬停变色。

    Color fillColor = (Color){45, 55, 70, 230}; // 默认按钮底色。
    Color lineColor = (Color){170, 180, 195, 220}; // 默认边框颜色。
    Color textColor = LIGHTGRAY; // 默认文字颜色。

    if (selected)

```

```

{
    fillColor = (Color){60, 120, 80, 240}; // 被选中的按钮用绿色。
    lineColor = (Color){230, 230, 160, 255}; // 被选中的按钮边框更亮。
    textColor = WHITE; // 被选中的按钮文字更亮。
}
else if (mouseOver)
{
    fillColor = (Color){65, 75, 92, 240}; // 鼠标悬停时略微变亮。
    textColor = WHITE;
}

DrawRectangleRec(button, fillColor);

DrawRectangleLines(
    (int)button.x,
    (int)button.y,
    (int)button.width,
    (int)button.height,
    lineColor);

int fontSize = 22; // 按钮文字大小。
int textWidth = MeasureText(text, fontSize); // 用于计算水平居中。
int textX = (int)(button.x + (button.width - textWidth) / 2); // 文字 x 居中。
int textY = (int)(button.y + (button.height - fontSize) / 2); // 文字 y 近似居
中。

```

中。

```

    DrawText(text, textX, textY, fontSize, textColor);
}

```

// 函数作用：绘制主菜单界面。

```

void DrawLevelSelectUI(const GameContext &context)
{

```

// 菜单界面先画背景，再画半透明菜单面板和按钮。

```

    DrawBackgroundImage(context);

```

```

    int menuX = GetMenuX(); // 菜单面板左上角 x。

```

```

    DrawRectangle(
        menuX,
        MENU_Y,
        MENU_WIDTH,
        MENU_HEIGHT,
        (Color){10, 15, 30, 200});

```

```

    DrawRectangleLines(
        menuX,
        MENU_Y,
        MENU_WIDTH,
        MENU_HEIGHT,
        (Color){180, 180, 200, 160});

```

```

    const char *title = "SNAKE MENU"; // 菜单标题。

```

```

    int titleSize = 72; // 标题字号。

```

```

    int titleWidth = MeasureText(title, titleSize); // 标题宽度，用于居中。

```

```

    DrawText(title, (SCREEN_WIDTH - titleWidth) / 2, MENU_Y + 50, titleSize,
    YELLOW);

```



```

const char *mapLabel = "Choose Map";
int mapLabelSize = 32;
int mapLabelWidth = MeasureText(mapLabel, mapLabelSize);

DrawText(mapLabel, (SCREEN_WIDTH - mapLabelWidth) / 2, MENU_Y + 140,
mapLabelSize, WHITE);

for (int i = 0; i < LEVEL_COUNT; i++)
{
    // 关卡按钮和地图大小文字一起画。
    Rectangle button = GetLevelButtonRect(i); // 当前关卡按钮的位置。

    std::string buttonText = "Level " + std::to_string(i + 1); // 按钮显示的关卡
    名。

    DrawButton(
        button,
        buttonText.c_str(),
        context.settings.currentLevel == i + 1);

    int mapSize = GetMapSizeByLevel(i + 1); // 当前关卡对应的地图边长。

    std::string mapText =
        std::to_string(mapSize) +
        " x " +
        std::to_string(mapSize);

    int mapTextWidth = MeasureText(mapText.c_str(), 22); // 地图大小文字宽度，用
    于居中。

    DrawText(
        mapText.c_str(),
        (int)(button.x + (button.width - mapTextWidth) / 2),
        (int)(button.y + button.height + 8),
        22,
        LIGHTGRAY);
}

const char *speedLabel = "Speed";
int speedLabelSize = 32;
int speedLabelWidth = MeasureText(speedLabel, speedLabelSize);

DrawText(speedLabel, (SCREEN_WIDTH - speedLabelWidth) / 2, MENU_Y + 340,
speedLabelSize, WHITE);

DrawButton(GetSpeedMinusButtonRect(), "-", false);
DrawButton(GetSpeedPlusButtonRect(), "+", false);

std::string speedText = "Level " + std::to_string(context.settings.speedLevel);
// 当前速度等级文字。
int speedTextWidth = MeasureText(speedText.c_str(), 34); // 用于居中。

DrawText(speedText.c_str(), (SCREEN_WIDTH - speedTextWidth) / 2, MENU_Y + 408,
34, WHITE);

const char *diffLabel = "Difficulty Target";

```

```

int diffLabelSize = 32;
int diffLabelWidth = MeasureText(diffLabel, diffLabelSize);

DrawText(diffLabel, (SCREEN_WIDTH - diffLabelWidth) / 2, MENU_Y + 490,
diffLabelSize, WHITE);

DrawButton(GetDifficultyMinusButtonRect(), "-", false);
DrawButton(GetDifficultyPlusButtonRect(), "+", false);

std::string difficultyText =
    "Level " +
    std::to_string(context.settings.difficultyLevel) +
    " Target " +
    std::to_string(context.settings.targetScore);

int diffTextWidth = MeasureText(difficultyText.c_str(), 32);

DrawText(difficultyText.c_str(), (SCREEN_WIDTH - diffTextWidth) / 2, MENU_Y +
558, 32, WHITE);

std::string crazyText; // Crazy Mode 开关按钮文字。

if (context.settings.crazyMode)
{
    crazyText = "Crazy Mode: ON"; // 开启时显示 ON。
}
else
{
    crazyText = "Crazy Mode: OFF"; // 关闭时显示 OFF。
}

DrawButton(GetCrazyModeButtonRect(), crazyText.c_str(),
context.settings.crazyMode);
DrawButton(GetStartButtonRect(), "START", false);
}

// 函数作用：根据当前游戏状态绘制菜单、分数和状态提示。
void DrawUI(const GameContext &context)
{
    // 如果当前在菜单状态，整个 UI 就是菜单。
    if (context.game.status == LevelSelect)
    {
        DrawLevelSelectUI(context);
        return;
    }

    // 游戏中先画分数，再画一行详细信息。
    std::string scoreText = "Score: " + std::to_string(context.game.score); // 当前
分数文字。

    Color scoreColor = WHITE; // 默认分数颜色。

    // 吃到食物后，分数短暂变黄。
    if (context.effects.scoreFlashTimer > 0.0f)
    {
        scoreColor = YELLOW; // 吃到食物后短暂变黄。
    }
}

```

```
int scoreSize = 42; // 分数字号。
int scoreWidth = MeasureText(scoreText.c_str(), scoreSize); // 分数文字宽度，用于居中。

DrawText(scoreText.c_str(), (SCREEN_WIDTH - scoreWidth) / 2, 25, scoreSize,
scoreColor);

std::string highScoreText = "High Score: " + std::to_string(context.highScore);
// 最高分文字。

std::string targetText;

// 疯狂模式没有目标分数，所以显示 Infinite。
if (context.settings.crazyMode)
{
    targetText = "Target: Infinite"; // 疯狂模式没有通关目标分。
}
else
{
    targetText = "Target: " + std::to_string(context.settings.targetScore); //
普通模式显示目标分。
}

std::string levelText =
    "Level: " +
    GetLevelName(context) +
    " | Map: " +
    std::to_string(context.settings.gridSize) +
    " x " +
    std::to_string(context.settings.gridSize);

std::string modeText;

// modeText 会附带疯狂模式的当前状态提示。
if (context.settings.crazyMode)
{
    modeText = "Mode: Crazy"; // 疯狂模式文字。
}
else
{
    modeText = "Mode: Normal"; // 普通模式文字。
}

if (context.crazy.boostTimer > 0.0f)
{
    modeText += " | Boost"; // 当前正在加速。
}
else if (context.crazy.redFlashTimer > 0.0f)
{
    modeText += " | Warning"; // 当前红光预警。
}
else if (context.crazy.greenFlashTimer > 0.0f)
{
    modeText += " | Recover"; // 当前绿光恢复提示。
}
```

```

std::string infoText =
    highScoreText +
    "    |    " +
    targetText +
    "    |    " +
    levelText +
    "    |    " +
    modeText;

int infoSize = 24; // 顶部信息字号。
int infoWidth = MeasureText(infoText.c_str(), infoSize); // 用于居中。

DrawText(infoText.c_str(), (SCREEN_WIDTH - infoWidth) / 2, 78, infoSize,
LIGHTGRAY);

std::string speedText =
    "Speed Level: " +
    std::to_string(context.game.score / SPEED_UP_SCORE + 1); // 分数越高，显示的
速度等级越高。

int speedWidth = MeasureText(speedText.c_str(), infoSize);

DrawText(speedText.c_str(), (SCREEN_WIDTH - speedWidth) / 2, 110, infoSize,
LIGHTGRAY);

// 屏幕左下角显示操作提示。
DrawText("WASD / Arrow Keys: Move", 20, SCREEN_HEIGHT - 120, 20, LIGHTGRAY);
DrawText("M: Back to Menu", 20, SCREEN_HEIGHT - 85, 20, LIGHTGRAY);
DrawText("Space: Pause | R: Restart | Esc: Quit", 20, SCREEN_HEIGHT - 45, 20,
LIGHTGRAY);

if (context.game.status == Paused)
{
    DrawCenterMessage("PAUSED", "Press Space to continue", YELLOW);
}

if (context.game.status == GameOver)
{
    std::string resultText =
        "Final Score: " +
        std::to_string(context.game.score) +
        " | R restart | M menu";

    DrawCenterMessage("GAME OVER", resultText.c_str(), RED);
}

if (context.game.status == Win)
{
    std::string resultText =
        GetLevelName(context) +
        " Clear | Score: " +
        std::to_string(context.game.score) +
        " | M menu";

    DrawCenterMessage("YOU WIN", resultText.c_str(), GREEN);
}

```

```

if (!context.game.started && context.game.status == Playing)
{
    const char *message = "Press WASD or Arrow Keys to start";
    int fontSize = 25; // 开始提示字号。
    int textWidth = MeasureText(message, fontSize); // 用于居中。

    DrawText(
        message,
        (SCREEN_WIDTH - textWidth) / 2,
        SCREEN_HEIGHT / 2 + 200,
        fontSize,
        (Color){200, 200, 200, 180});
}
}

```

// 函数作用：绘制暂停、失败、胜利时居中的提示面板。

```

void DrawCenterMessage(const char *title, const char *subtitle, Color titleColor)
{
    // 暂停、失败、胜利都用同一个居中面板。
    int boxWidth = 520; // 中央提示框宽度。
    int boxHeight = 170; // 中央提示框高度。
    int boxX = (SCREEN_WIDTH - boxWidth) / 2; // 提示框水平居中。
    int boxY = SCREEN_HEIGHT / 2 - 125; // 提示框略高于屏幕中心。

    DrawRectangle(boxX, boxY, boxWidth, boxHeight, (Color){0, 0, 0, 150});
    DrawRectangleLines(boxX, boxY, boxWidth, boxHeight, (Color){220, 220, 220,
140});

    int titleSize = 58; // 主标题字号。
    int subtitleSize = 26; // 副标题字号。
    int titleWidth = MeasureText(title, titleSize); // 主标题宽度，用于居中。
    int subtitleWidth = MeasureText(subtitle, subtitleSize); // 副标题宽度，用于居
中。

    DrawText(title, (SCREEN_WIDTH - titleWidth) / 2, boxY + 30, titleSize,
titleColor);
    DrawText(subtitle, (SCREEN_WIDTH - subtitleWidth) / 2, boxY + 105,
subtitleSize, LIGHTGRAY);
}

```